4.1 Compare ArrayList, Vector, LinkedList : underlying structure, synchronization, insert/delete vs random access performance.

All three are classes in the Java Collection Framework used to store multiple objects.

Feature	Arraylist	Vector	Linkedlist
Structure	Dynamic Array	Dynamic Array	Doubly linked list
Synchronization	Not synchronized	Synchronized	Not synchronized
Random access	Fast, O(1)	Fast, O(1)	Slow, O(n)
Insert/delete middle	Relatively slower	Relatively slower	Faster if node position in known
Performance	Generally fast	Slower due to synchronization	Good for frequent insertion/deletion
Modern Usage	Very common	Less common	Common for linked list operations

ArrayList - uses a dynamic array. It is good when we frequently access elements using indexes.

```
ArrayList<String> list = new ArrayList<>();
```

Vector - is also a dynamic array but its methods are synchronized. Therefore, it is generally slower than Arraylist.

LinkedList - stores elements using linked nodes. Insertion and deletion can be efficient, but random access is slower.

4.2 Explain set and its implementations HashSet, LinkedHashSet, TreeSet. How does TreeSet maintain order?

Set - is a collection that does not allow duplicate elements.

```
Set<String> names = new HashSet<>();
```

The major implementations are -

1. HashSet -
   - Does not allows duplicates.
   - Does not guarantee insertion order.
   - Uses hashing internally.
   - Usually provides fast operations.

```
HashSet<String> set = new HashSet<>();
```

2. LinkedHashSet -
   - Does not allows duplicates
   - Maintains insertion order
   - Uses hashing with linked structure

```
LinkedHashSet<String> set = new LinkedHashSet<>();
```

3. TreeSet -
   - Does not allow duplicates.
   - Maintains elements in sorted order.
   - Uses a tree-based structure
   - Normally uses natural ordering or a supplied Comparator.

```
TreeSet<String> set = new TreeSet<>();
```

Comparison -

set	Duplicate	Order
HashSet	No	No guaranteed order
LinkedHashSet	No	Insertion order
TreeSet	No	sorted order

How does TreeSet maintain order -

TreeSet is based on a balanced search tree (Red-Black tree) and keeps elements sorted according to their natural ordering or Comparator.

4.3 Store 5 student names in an Arraylist (print with enhanced for-loop) and in a Treeset (print, observe sorting). Comment on ordering difference.

```
import java.util.ArrayList;
import java.util.TreeSet;

public class Main {
  public static void main (String[] args) {
    ArrayList <String> students = new ArrayList <>();
    students.add ("Rahim");
    students.add ("Karim");
    students.add (" Rubi ");
    students.add (" Rahim");
    students.add (" Tania");
    System.out.println ("ArrayList:");
    for (String name : students) {
        System.out.println (name);
    }
  TreeSet <String> sortedStudents = new TreeSet<>();
    sortedStudents.add (" Rahim");
    sortedstudents.add ("Karim");
    sortedstudents.add (Rubi");
    SortedStudents.add("Raham");
    sortedstudents.add (" Tania");
```

```
System.out.println("\n Treeset:");
for (String name : sortedStudents) {
    System.out.println(name);
}}
```

Output -

Arraylist :

Rahim
karim
Rubi
Rahan
Nadda

Treeset :

Rahim
karim
Rubi
Raham
Nadia.

5.1 List ≥ 3 ways to implement multithreading in Java (extends Thread, implements Runnable, ExecutorService/Callable). Which is prefferred and why?

Multithreading - means executing multiple threads concurrently within a program. (A thread is a lightweight, independent unit of execution inside a program.)

There are several ways to create/manage threads in Java.

1. Extending Thread - create a class that extends Thread and override run().

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
```

2. Implementing Runnable - create a class implementing Runnable.

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}
public class Main {
    public static void main(String[] ads) {
        new Thread(new MyTask()).start();
    }
}
```

3. ExecutorService - manages a pool of threads and is useful for largere applications.

```
class MyTask implements Runnable {
@Override
 public void run() {
 for(int i=1 ; i<=5 ; i++){
 System.out.println(Thread.currentThread().getName() +
                                  is running : " +i);
 try{
    Thread.sleep(500);
 } catch (InterruptedException e){
    system.out.println("Thread was interrupted"); }}
```

Callable - is useful when a thread needs to return a result or throw an exception.

Which preferred - For simple tasks, Runnable is often prefferced over extending Thread because Java supports single inheritance. If we extend Thread we cannot extend another class.

For largere applications, ExecutorService is generally preferred because it manages thread creation and reuse effeiently.

5.2 Two Threads printing 1-5 concurrently - one via extends Thread, one via implements Runnable. Show how both are started from main().

```
class ThreadOne extends Thread {
  @Override
    public void run () {
     for (int i = 1 ; i <= 5 ; i++) {
       System.out.println ("Thread One:" + i);
    }
  }
}
class ThreadTwo implements Runnable {
@Override
 public void run() {
  for (int i = 1 ; i <=5 ; i++) {
   System.out.println("Thread Two:"+ i);
}}
}
public class Main {
  public static void main (String[] args) {
   ThreadOne t1 = new ThreadOne();
   ThreadTwo task = new ThreadTwo();
   Thread t2 = new Thread (task);
    t1.start();
    t2.start();
}
}
```

Output -

```
Thread One : 1
Thread Two : 1
Thread One : 2
Thread Two : 2
Thread Two : 3
Thread One : 3
Thread One : 4
Thread Two : 4
Thread One : 5
Thread Two : 5
```

Here, we used : t1.start();
t2.start();

not t1.run()
t2.run() ; because start() creates a new thread and then calls run().

5.3 Custom check exception InvalidRadiusException. Circle constructor throws it if radius < 0. Main reads radius, handles via try-catch, else print area.

```
import java.util.Scanner;
class InvalidException extends Exception{
  public InvalidRadiusException (String message){
    super (message);
  }}
class Circle {
  private double radius;
  public Circle (double radius) throws InvalidRadiusException{
  if (radius <0) {
    throw new InvalidRadiusException (
      "Radius cannot be negetive."
                                    }; }
    this.radius = radius;
  }
  public double getArea() {
    return Math.PI * radius * radius;
                                    } }
public class Main {
  public static void main (String[] args) {
    Scanner input = new Scanner (System.in);
    System.out.print ("Enter radius: ");
```

```
double radius = input.nextDouble();
try {
    Circle c = new Circle(radius);
    System.out.println("Area = " + c.getArea());
} catch (InvalidRadiusException e) {
    System.out.println("Error: " + e.getMessage());
}
}
}
```

Input :- Enter Radius : 5

Output :- Area = 78.53981633974483

Input : Enter Radius : -5

Output : Error : Radius cannot be negative.

6.1 Steps to connect Java to MySQL / Oracle via JDBC. Name key classes (DriverManager, Connection, PreparedStatement, ResultSet).

JDBC (Java Database Connectivity) is an API that allows Java programs to communicate with database such as MySQL and Oracle.

Main steps to connect Java to MySQL -

Step-1 — Add JDBC Driver —

Add the MySQL Connector/J or Oracle JDBC driver to the project.

Step-2 - Load/Register Driver-

Modern JDBC drivers are usually automatically discovered, but traditionally -

```
Class.forName ("com.mysql.cj.jdbc.Drive");
```

Step 3 - Create Connection —

```
Connection con = DriverManager.getConnection(
    url, username, password );
```

Step 4 - Create PreparedStatement

```
preparedStatement ps = con.prepareStatement(
    "INSERT INTO students VALUES(?,?,?)" );
```

Step 5 - Execute SQL -

```
ps.executeUpdate();
```

Step 6 - Read Result -

For SELECT :

```
ResultSet rs = ps.executeQuery();
```

Step 7 - close Resources

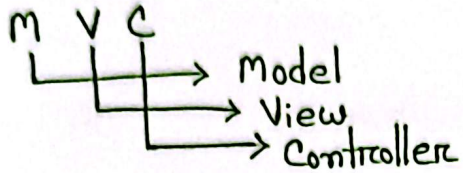
Close ResultSet, PreparedStatement and Connection

Important JDBC classes -

- DriverManager → create database connection
- Connection → represents connection with database
- PreparedStatement → executes parameterized SQL
- ResultSet → stores result of SELECT

6.2 Explain the MVC pattern and how a JDBC app's classes map to Model/View/Controller.

MVC stands for :



, MVC separates an application into three parts.

Model - represents data and business information.

Example :
```
class Student {
    int id;
    String name;
    double cgpa; }
```

View - The view is responsible for user interface / input/output.

Example : • JSP • JavaFx • HTML

Controller - The Controller receives user requests and controls the application flow.

For JDBC applications, a DAO (Data Access Object) is often used to perform database operations.

Example : View
↓
Controller / DAO
↓
Model / Database

JDBC MVC mapping -

MVC	Example
Model	student.java
View	Main.java / JSP
Controller/DAO	studentDAO.java
Database	MySQL/Oracle.

Advantages -

MVC makes the application organized, maintainable, reusable and easier to debug.

6.3 MVC-style code for student: Model (Student), Controller / DAO (StudentDAO.insert() using PreparedStatement), View (Main) collecting input and calling the controller.

Model-

```
class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student (int id, String name, double cgpa){
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;      }
    public int getID(){
        return id;    }
    public String getName(){
        return name;    }
    public double getCgpa(){
        return cgpa;
    }}
```

DAO/Control

```
import java.sql.*;

class StudentDAO {
    private String url =
        "jdbc:mysql://localhost:3306/student_db";
    private String username = "root";
    private String password = "1234";
    public void (Student s) { // method
    String sql = "INSERT INTO students (id,
        name, cgpa) VALUES (?, ?, ? )";
  try (
    Connection con = DriverManager.getConnection(
        url, username, password );
    PreparedStatement ps = con.preparedStatement(sql)
  ) {
    ps.setInt(1, s.getId());
    ps.setString (2, s.getName());
    ps.setDouble (3, s.getCgpa());
    ps.executeUpdate();
    System.out.println("Student inserted.");
```

```
} catch (SQLException e){
    System.out.println (e.getMessage ());
}}}
```

View / Main -

```
import java.util.Scanner;

public class Main {
    public static void main (String[] args) {
    Scanner input = new Scanner (System.in);
    System.out.print ("Enter ID: ");
    int id = input.nextInt();
    input.nextLine();
    System.out.print ("Enter Name: ");
    String name = input.nextLine();
    System.out.print("Enter CGPA: ");
    double cgpa = input.nextDouble();
    Student student = new Student (id, name, cgpa);
    StudentDAO dao = new StudentDAO();
    dao.insert (student);
}}
```

Flow:

Main / View → Student Model ↓

MySQL Database ← Prepared Statement